Bruise Segmentation Pipeline

Function-by-Function Reference

BRUISE_SEGMENTATION_PROJECT

Generated for the 5-model scope: SegFormer B2 teacher, SegFormer B0 direct, SegFormer B0 distilled, YOLO26n-sem direct, YOLO26n-sem distilled

Scope of this document

This reference covers every file remaining in `pipeline/` and `scripts/` after the ALS / fairness-distillation / ITA / skin-tone files were moved to `out_of_scope/` (they are not documented here, since current project focus is only the 5 models listed above). Part I covers the shared `pipeline/` modules; Part II covers the numbered pipeline scripts and standalone utility scripts in `scripts/`, in the order they are actually run.

Contents

I Shared Pipeline Modules (`pipeline/`) 3

1 pipeline/data.py 3

2 pipeline/io_utils.py 5

3 pipeline/models.py 6

4 pipeline/batch_finder.py 8

5 pipeline/losses.py 9

6 pipeline/mask_utils.py 10

7 pipeline/metrics.py 11

8 pipeline/metrics_extended.py 12

9 pipeline/scheduler.py 15

10 pipeline/trainer.py 16

11 pipeline/yolo_threshold_temp.py 18

12 pipeline/benchmark_640.py 20

II Pipeline Scripts (scripts/) 23

13 scripts/00_build_split.py 23

14 scripts/01_train_segformer_b2_teacher.py 23

15 scripts/02_calibrate_teacher.py 23

16 scripts/03_train_segformer_b0_direct.py 24

17 scripts/04_optuna_alpha_segformer_b0.py 24

18 scripts/05_train_segformer_b0_distilled.py 25

19 scripts/06_train_yolo_sem_direct.py 25

20 scripts/07_optuna_alpha_yolo_sem.py 26

21 scripts/07b_threshold_yolo_direct.py 26

22 scripts/07c_threshold_yolo_distilled.py 27

23 scripts/08_train_yolo_sem_distilled.py 27

24 scripts/09_evaluate_test.py 28

25 scripts/09b_evaluate_yolo_temp_scaled.py 28

26 scripts/10_consolidate_results.py 29

27 scripts/10_track_a_evaluate.py 29

28 scripts/11_track_b_evaluate.py 30

29 scripts/24_consolidate_all_results.py 31

30 scripts/26_migrate_legacy_segformer_checkpoints.py 32

31 scripts/27_evaluate_test_640_models_01_to_11.py 33

32 scripts/29_benchmark_inference_all_models.py 34

33 scripts/31_benchmark_640_local.py 35

34 scripts/fix_local_manifest_paths.py 36

35 scripts/visualize_segformer_stages.py 36

36 scripts/yolo_wl_audit_v1.py 37

Part I

Shared Pipeline Modules (`pipeline/`)

1 pipeline/data.py

Defines the dataset class and preprocessing transform used by every training, calibration, threshold-search, and evaluation step in the project — for both SegFormer and YOLO. Centralizing this in one file guarantees that whatever operating point (threshold, temperature) gets calibrated on the validation set is always applied to the same input distribution at test time.

`_find_col(df: pd.DataFrame, candidates: list[str]) -> str | None`

Searches a manifest DataFrame's columns for the first name in `candidates` that exists, trying an exact match first and then a case-insensitive match. Exists because different manifest CSVs in this project use slightly different column names for the same concept (e.g. `mask_path` vs. `majority_mask_path`), and this lets one normalization function handle all of them without hardcoding a single schema.

`normalize_manifest(df: pd.DataFrame) -> pd.DataFrame`

Standardizes any manifest DataFrame to guarantee `image_path`, `mask_path`, `stem`, and `subject` columns exist, deriving `stem` from the image filename and `subject` from the stem (splitting on underscore/hyphen) if not already present. Raises `RuntimeError` with the full list of available columns if no image or mask column can be found — failing loudly immediately rather than crashing confusingly deep inside a training loop. Every downstream script calls this before touching a manifest, so schema differences between manifests are handled in exactly one place.

`load_train_val_split(csv_path: str) -> tuple[pd.DataFrame, pd.DataFrame]`

Loads and normalizes a split CSV, then splits it into train/val DataFrames based on its `split` column, raising `RuntimeError` if that column is missing (telling the caller to run `00_build_split.py` first).

`load_fixed_test(csv_path: str) -> pd.DataFrame`

Loads and normalizes the fixed held-out test manifest. A thin wrapper so every script that reads the test set goes through the same normalization path as train/val.

`get_augmentation(training: bool, img_h: int, img_w: int) -> A.Compose`

Builds the Albumentations transform pipeline: resize to `img_h` x `img_w`, ImageNet mean/std normalization, convert to tensor. When `training=True`, adds horizontal/vertical flips, brightness/contrast jitter, hue/saturation/value jitter, and Gaussian noise; when `False` (validation, test, and every benchmark/inference use in this project), it is the deterministic resize+normalize+to-tensor pipeline only. This is the single function reused by `pipeline/benchmark_640.py` for both SegFormer and YOLO inference, guaranteeing the benchmark sees the same input distribution the model was calibrated on.

`class BruiseDataset(Dataset)`

The shared PyTorch `Dataset` implementation used by every SegFormer training/eval script and by the YOLO threshold/temperature search. `__init__(df, img_h, img_w, training=False)` stores the manifest DataFrame and builds its transform via `get_augmentation`. `__len__()` returns the row count. `__getitem__(idx)` reads the image with OpenCV, converts BGR to RGB, reads the mask as grayscale and binarizes it as `mask > 0` (not `mask == 255`) — robust to masks whose foreground pixel value is not exactly 255 (e.g. anti-aliased or class-index masks) — applies the transform to both image and mask jointly (so any spatial augmentation stays aligned between them), and returns `(image_tensor, mask_tensor, stem, image_path, mask_path)`. Returning the stem and paths alongside the tensors lets every evaluation script attribute a metric row back to a specific source image without a second lookup.

2 pipeline/io_utils.py

Foundation utilities shared by every script: YAML/JSON I/O, directory creation, structured logging, and pre-flight config validation. Centralizing these means every script fails with an identical, clear error message for a missing config or bad path, rather than each script inventing its own (inconsistent) error handling.

`load_yaml(path: str | Path) -> dict`

Loads a YAML file, raising `FileNotFoundError` with an actionable message (run from the project root, or pass an absolute path) if it does not exist — rather than the bare, less-clear error `open()` itself would raise. Coerces an empty file to `{}` so callers can unconditionally use `cfg.get(...)` without an `AttributeError`.

`ensure_dir(path: str | Path) -> Path`

Creates a directory and all missing parents (`parents=True, exist_ok=True`), returning the `Path` so callers can chain it, e.g. `csv_path = ensure_dir(run_dir / "eval") / "results.csv"`.

`save_json(path: str | Path, payload: dict) -> None`

Serializes a dict to a pretty-printed (`indent=2`) JSON file, creating parent directories first. Used for `run_config.json` and `temperature.json` so downstream scripts can read back exact hyperparameters without re-parsing CLI arguments.

`setup_logging(run_dir: Path | str | None = None, level: int = logging.INFO) -> logging.Logger`

Configures and returns the shared `"pipeline"` logger with a consistent timestamped format, writing to stdout always and additionally to `run_dir/run.log` if a `run_dir` is given (append mode, so a resumed run keeps its history). Clears any existing handlers first so calling this twice in one process never duplicates log lines.

`validate_paths(paths: dict) -> None`

Pre-flight check for `configs/paths.yaml`: asserts every required key is present (collecting *all* missing keys before raising, not just the first, so a user fixes every typo in one pass), then checks that every required *input* path (manifests, pretrained weights, YOLO weights) actually exists on disk, raising `RuntimeError` listing every missing path at once. Output-only paths (like `project_root`) are not checked here since scripts create those themselves.

`validate_cfg(cfg: dict) -> None`

Pre-flight check for `configs/common_train.yaml`: asserts every required key is present, then sanity-checks numeric ranges (e.g. `epochs` in $[1, 10000]$, `patience` $\leq$ `epochs`, learning rates in $(0, 1)$, a non-empty `thresholds` list) so a unit-error typo (like a learning rate of 6 instead of 0.00006) is caught before any GPU time is spent, not discovered hours into a training run.

3 pipeline/models.py

Defines the SegFormer model wrapper and the checkpoint/threshold loading functions used by every SegFormer training and evaluation script (including both benchmark scripts).

`build_segformer(pretrained: str, num_labels: int = 1) -> nn.Module`

Loads a HuggingFace `SegformerForSemanticSegmentation` from a local pretrained directory, with `num_labels=1` (binary bruise segmentation) and `ignore_mismatched_sizes=True` (since the pretrained checkpoint's original 1000-class ImageNet decode head does not match a 1-class head — that head is discarded and randomly reinitialized, while the backbone/encoder weights load normally).

`class SegformerWrapper(nn.Module)`

Wraps the HuggingFace model, exposing `.backbone` and `.decode_head` as properties so `pipeline/scheduler.py` can give them different learning rates. `forward(x)` runs the model and, if the returned logits are not already at the input's spatial resolution, upsamples them (bilinear) to match — this is why `SegFormer640Adapter` in `benchmark_640.py` never needs a separate resize step. `gradient_checkpointing_enable()` turns on gradient checkpointing on the encoder if available, silently doing nothing if the underlying HuggingFace version does not expose that attribute.

`count_params(model: nn.Module) -> tuple[int, int]`

Returns `(total_parameters, trainable_parameters)` by summing `.numel()` over all parameters (the second sum filtered to `requires_grad`).

`load_segformer_threshold(run_dir: Path) -> float`

Reads a SegFormer run's `threshold_search.csv` (written by the training loop's validation-set sweep), sorts by `mean_dice` descending, and returns the top row's threshold. This is always read from the **validation** set, never recomputed on test — fitting a threshold on the same data used to report accuracy would leak information and inflate the reported score.

`_pick_matching_checkpoint(run_dir: Path, model: nn.Module) -> tuple[Path, dict]`

Returns whichever of `best_model.pt` or `best_model.pre_migration_backup.pt` actually has parameter names matching the currently-installed `transformers` version. This exists because `26_migrate_legacy_segformer_checkpoints.py` rewrote some checkpoints' internal parameter names to match the `transformers` version ORC had at one point, keeping the pre-migration original as a safety backup — but a different machine's `transformers` install may match one file or the other, so both are loaded and compared by key set (not by tensor values, which is cheap since both are already read into memory) before either is actually used. Raises `RuntimeError` naming both candidates if neither matches, pointing at script 26's docstring for how to add a new remapping rule.

`load_segformer_model(model_name: str, pretrained_key: str, paths: dict, device: torch.device) -> tuple[nn.Module, float, Path]`

The single function every SegFormer-consuming script (training scripts' `load_teacher`, script 29, and `benchmark_640.py`) uses to load a trained run: resolves the threshold via `load_-`

`segformer_threshold`, builds a fresh model via `build_segformer`/`SegformerWrapper`, and loads whichever checkpoint file `_pick_matching_checkpoint` determines is compatible. Returns `(model, threshold, checkpoint_path)` so callers can both run inference and record exactly which checkpoint produced a given result.

4 pipeline/batch_finder.py

Automatically probes the largest training batch size that fits in available GPU memory, so a single hardcoded batch size does not under-utilize the GPU for a small model or crash with out-of-memory on a large one (e.g. the teacher, which additionally must carry a frozen teacher model through every forward pass during distillation).

```
find_optimal_micro_batch(model: nn.Module, img_h: int, img_w: int, device: torch.device,
effective_batch: int = 8, target_hi: float = 0.75, amp: bool = True, max_probe: int = 32,
teacher_fn=None) -> tuple[int, int, float]
```

Probes increasing batch sizes (doubling each time: 1, 2, 4, 8, …) with a real forward + backward + optimizer step on random dummy data at the model's actual training resolution, stopping when either an out-of-memory error occurs or the peak VRAM fraction exceeds `target_hi` (default 75%, leaving headroom for fragmentation and the real data pipeline). It measures *peak* memory via `torch.cuda.max_memory_reserved()` (reset before every probe) rather than current memory, because current memory under-reports the true high-water mark once autograd has already freed forward-pass activations. If `teacher_fn` is supplied (a distillation run), it is invoked on every probe batch too, since real training calls the teacher every step and its memory footprint must be included or the chosen batch size would out-of-memory the instant real training starts. Returns `(micro_batch, accum_steps, vram_fraction)`, where `accum_steps = effective_batch // micro_batch` so gradient accumulation restores the desired effective batch size regardless of what the hardware can sustain directly. On a CPU-only machine, probing is skipped entirely (there is no VRAM to probe) and it simply returns `min(effective_batch, max_probe)` as the micro-batch.

5 pipeline/losses.py

Defines the two loss functions used across every SegFormer training run: a combined Dice+BCE loss for direct (non-distilled) training, and a distillation loss blending ground truth with the teacher's soft probability map.

`class DiceBCELoss(nn.Module)`

`forward(logits, target)` computes binary cross-entropy directly on raw logits (numerically stable, versus applying sigmoid then BCE separately), plus a soft Dice loss (sigmoid probabilities, $2 \cdot |P \cap G| / (|P| + |G| + \text{smooth})$, with a Laplace-smoothing constant to avoid division by zero on empty masks), and returns `bce + (1 - mean_dice)`. BCE gives stable gradients everywhere including on empty masks, while Dice directly optimizes the overlap metric that is ultimately reported — compensating for BCE's weakness on a small, class-imbalanced foreground (bruises against a large background).

`class DistillSegLoss(nn.Module)`

Implements `loss = alpha * DiceBCELoss(student, GT) + (1 - alpha) * BCE(student, teacher_prob)` — the same fusion formula used everywhere distillation appears in this project, so the weighting convention stays consistent across SegFormer and YOLO distillation rather than being tuned per-script. `__init__(alpha=0.75)` stores the mixing weight (ground truth dominates 3:1 by default) and an internal `DiceBCELoss` instance for the supervised term. `forward(logits, gt, teacher_prob)` computes the supervised Dice+BCE term against ground truth, plain BCE-with-logits against the teacher's (already temperature-scaled, sigmoid-applied) soft probability map, and returns the alpha-weighted sum.

6 pipeline/mask_utils.py

Small mask-related I/O and conversion helpers used by the YOLO training/ evaluation scripts and by the teacher soft-labeling code that feeds YOLO distillation.

`link_or_copy(src: Path, dst: Path) -> None`

Creates `dst`'s parent directories, then tries to symlink `dst` to `src`; if symlinking fails (e.g. lacking privilege on Windows), falls back to a real file copy. This lets the pipeline build YOLO-format dataset directories cheaply via symlinks where supported, without duplicating large image files on disk, while still working (just using more disk space) where symlinks are not available. A no-op if `dst` already exists, so it is safe to call repeatedly across re-runs.

`read_gt_mask(path: str) -> np.ndarray`

Reads a grayscale mask via OpenCV, raising `RuntimeError` naming the path if unreadable (rather than a confusing downstream `None` crash), and binarizes as `mask > 0` — robust to masks stored with different foreground pixel values.

`teacher_prob_for_image(model, temperature: float, img_bgr: np.ndarray, img_h: int, img_w: int, device: torch.device) -> np.ndarray`

Generates a teacher soft-probability map for one full-resolution clinical photo: converts BGR to RGB, *downsamples* to the teacher's trained resolution (640x640), normalizes with ImageNet mean/std, runs the teacher under `no_grad`, applies temperature-scaled sigmoid, then *upsamples the resulting probability map* back to the photo's native resolution (clipped to $[0, 1]$ against interpolation overshoot). Downsampling before inference and upsampling only the result — rather than running the teacher at native resolution — is necessary because SegFormer's self-attention memory scales with image area, and running at full clinical-photo resolution would out-of-memory.

`save_semantic_class_mask(mask01: np.ndarray, path: Path) -> None`

Writes a binary mask (0 = background, 1 = bruise) as an 8-bit image, matching Ultralytics' semantic-segmentation label format. Raises `RuntimeError` on write failure so a missing/corrupt label file is caught immediately rather than failing mysteriously during YOLO training.

`yolo_sem_pred_mask(result, shape: tuple[int, int]) -> np.ndarray`

Extracts a binary prediction mask from an Ultralytics semantic-segmentation result object. Ultralytics' semantic task exposes neither `.masks` (instance segmentation) nor `.probs` (classification) — the dense per-pixel class map instead lives at `result.semantic_mask.data`. This function pulls that tensor, thresholds it to isolate the bruise class, and resizes (nearest-neighbor, to avoid blended fractional class values) to the requested output shape. Returns an all-zero mask if the expected attribute is missing, so a batch evaluation loop can continue past an unexpected result rather than crashing.

7 pipeline/metrics.py

*The **original** pixel-level segmentation metrics module (Dice, IoU, precision/recall, per-image and aggregate summaries), used by scripts 06–09. Deliberately left unmodified even after a known correctness issue was found (see `metrics_extended.py`) specifically so that the numbers those scripts already produced and recorded remain reproducible and comparable to themselves over time.*

`dice_np(pred: np.ndarray, gt: np.ndarray) -> float`

Computes $2 \cdot |P \cap G| / (|P| + |G|)$, returning 1.0 when the denominator is zero (both masks empty). Note: this returns 1.0 even when `pred` is non-empty and `gt` is empty — a false positive on a clean image — which `metrics_extended.py`'s `dice_safe` identifies as a bug and fixes; this file is kept exactly as-is anyway, for the reproducibility reason stated above.

`iou_np(pred: np.ndarray, gt: np.ndarray) -> float`

Computes $|P \cap G| / |P \cup G|$, returning 1.0 by convention when the union is zero (both masks empty, a true negative — correctly treated as perfect agreement, unlike the Dice case above).

`precision_recall_np(pred: np.ndarray, gt: np.ndarray) -> tuple[float, float, int, int, int]`

Computes true/false positives and false negatives via boolean operations. Precision defaults to 1.0 if there were no positive predictions at all; recall defaults to 1.0 if the ground truth has no positive pixels at all (both vacuous-truth conventions). Returns all five raw values so callers can compute dataset-level aggregate precision/recall without recomputing pixel counts.

`compute_image_row(pred: np.ndarray, gt: np.ndarray, stem: str) -> dict`

Aggregates Dice, IoU, precision, recall, raw pixel counts, and a predicted-to-ground-truth positive-pixel ratio (an over/under-segmentation indicator, set to `NaN` when ground truth is empty) into one dictionary row for a single evaluated image, keyed by `stem` for later joining back to source images.

`summarize(rows: list[dict]) -> dict`

Turns a list of per-image rows into dataset-level aggregates: image count, mean/median Dice and IoU, counts/rates of zero-Dice images, counts/rates of "complete miss" images (model predicted nothing at all where a bruise existed — a stricter failure mode than zero Dice alone), mean precision/recall, and mean predicted-to-ground-truth ratio (with infinite values replaced by `NaN` first so they cannot corrupt the mean). Returns an empty dict if given no rows.

8 pipeline/metrics_extended.py

*A newer, **additive** evaluation module used by scripts 09, 10, 11, and 27, built for a more rigorous benchmark. Its own docstring explains why it is a separate file rather than a fix to `metrics.py` in place: scripts 06–09 already produced committed results using the original functions, and changing them there would silently change already-recorded numbers.*

`_bool_masks(pred, gt) -> tuple[np.ndarray, np.ndarray]`

Casts both arrays to boolean in one shared place, since every metric function in this module needs boolean arrays.

`_pixel_counts(pred_b, gt_b) -> tuple[int, int, int, int, int]`

Computes predicted-positive count, ground-truth-positive count, true positives, false positives, and false negatives in one pass, since Dice/IoU/precision/ recall all need the same five numbers.

`dice_safe(pred, gt) -> float`

The corrected four-case Dice fixing `metrics.py`'s bug: both masks empty → 1.0 (a true negative, correctly rewarded); exactly one empty → 0.0 (properly penalizing false positives and complete misses, unlike the original); both non-empty → standard Dice.

`iou_safe(pred, gt) -> float`

Same as `iou_np` (its empty-mask edge case was not actually broken); exists mainly for naming symmetry with `dice_safe`.

`_boundary(mask: np.ndarray) -> np.ndarray`

Extracts a mask's outer boundary ring by eroding it by one pixel and XOR-ing against the original — the standard morphological boundary definition used in the Surface Dice literature, chosen to match that literature rather than an ad hoc heuristic.

`_distance_from_boundary(mask: np.ndarray) -> np.ndarray`

Computes a per-pixel Euclidean distance transform to the nearest boundary pixel, in $O(n)$ time for an n-pixel image. Returns all-infinity if the mask has no boundary (i.e. is empty), designed to propagate into downstream `NaN`/ zero checks rather than raising.

`surface_dice(pred, gt, delta: float = 2.0) -> float`

Implements the boundary-tolerant Surface Dice metric (Nikolov et al. 2018): what fraction of each predicted/ground-truth boundary lies within `delta` pixels of the other boundary. Standard Dice weights every interior pixel equally, but for bruise segmentation a small boundary misalignment is clinically acceptable while completely missing the bruise edge is not — Surface Dice specifically measures boundary tracking, independent of interior overlap. Default `delta=2.0` pixels, per supervisor specification.

`asd_px(pred, gt) -> float`

Average Surface Distance in pixels: the mean boundary-to-boundary distance in both directions, averaged. Where Surface Dice only answers "is the boundary close enough," ASD quantifies *how far off* it is on average, a continuous severity measure Surface Dice alone cannot give.

`hd95_px(pred, gt) -> float`

The 95th-percentile (not the classical 100th-percentile/maximum) Hausdorff Distance, taken as the max of both directions' 95th percentiles. Using the 95th percentile rather than the true maximum makes the metric robust to a single outlier boundary pixel (e.g. one disconnected false-positive fragment) that would otherwise dominate and make the classical Hausdorff Distance uninterpretable.

`_safe_precision(tp, fp) -> float`

/

`_safe_recall(tp, fn) -> float`

Same vacuous-truth conventions as `metrics.py`, explicitly noted to match scikit-learn's `zero_division=1` convention rather than an arbitrary choice.

`compute_image_row_extended(pred, gt, stem, surf_dice_delta=2.0) -> dict`

The extended, drop-in replacement for `compute_image_row`: adds the three boundary metrics above plus explicit `zero_dice` and `complete_miss` boolean flags (the latter meaning the model predicted nothing at all on an image that does contain a bruise).

`bootstrap_ci(values: np.ndarray, n_bootstrap: int = 2000, ci: float = 0.95, stat: str = "median", seed: int = 0) -> tuple[float, float]`

Computes a bootstrap confidence interval for the median or mean of a metric column via resampling with replacement and taking empirical percentiles. Bootstrapping is used instead of a parametric interval because per-image Dice scores are heavily skewed (many exact zeros from complete misses, many values near 1.0 from easy cases) rather than normally distributed, which is the field-standard justification for this method in medical-imaging metrics. `NaN` values are dropped before resampling; the seed is fixed for reproducibility across repeated runs.

`wilcoxon_compare(a: np.ndarray, b: np.ndarray) -> dict`

Runs a paired, two-sided Wilcoxon signed-rank test between two models' per-image scores on the *same* images, plus a rank-biserial correlation as an effect size. Wilcoxon is used instead of a t-test because Dice scores are non-normal; the pairing matters because each row of `a` and `b` is the same test image, making the samples dependent. The effect size is reported alongside the p-value because with a large test set, p-values alone can flag a trivially small difference as statistically significant. Returns `NaN` results with an explicit warning if fewer than 10 valid (finite, paired) samples remain, since the test is considered unreliable below that size.

`_aggregate_column(df: pd.DataFrame, col: str, n_bootstrap: int) -> dict`

For one metric column: median, IQR, mean, standard deviation, and a 95% bootstrap CI. IQR is reported as the statistically correct spread measure for skewed data; standard deviation is included

mainly because readers expect it.

`summarize_extended(rows: list[dict], n_bootstrap: int = 2000) -> dict`

The extended counterpart to `metrics.py`'s `summarize`: aggregates Dice, IoU, surface Dice, ASD, HD95, precision, and recall via `_aggregate_column`, plus zero-Dice/complete-miss counts and rates. Centralizing this in one function guarantees scripts 09, 10, 11, and 27 all report an identical set of columns, and a new metric added here automatically propagates to every calling script.

9 pipeline/scheduler.py

Implements the layer-wise learning-rate and warmup/polynomial-decay schedule for SegFormer fine-tuning, following the original SegFormer paper's recipe (Xie et al., 2021). The module's own docstring notes this is a deliberate departure from a constant-LR convention used elsewhere in the project for an earlier, unrelated ablation — this folder tests the paper's own recipe on its own terms.

`_is_no_decay(name: str, param) -> bool`

Flags parameters that should be excluded from weight decay: names containing "norm" or "bias," or any 1-D-or-less tensor (catching bias vectors and norm scale/shift parameters that might not literally have those substrings in their name). Applying weight decay to normalization or bias parameters is known to hurt rather than help training.

`build_param_groups(model, backbone_lr: float, head_lr: float, weight_decay: float) -> list[dict]`

Splits every trainable parameter into four optimizer groups — backbone/decay, backbone/no-decay, head/decay, head/no-decay — based on whether a parameter belongs to `model.backbone` versus `model.decode_head`, and whether `_is_no_decay` flags it. Backbone groups get `backbone_lr`, head groups get `head_lr`; no-decay groups get `weight_decay=0.0`. Frozen parameters are skipped entirely, and empty groups are filtered out (an empty group would error PyTorch's optimizer). This four-way split is what lets a single `AdamW(param_groups)` call express the paper's layer-wise-LR and selective-weight-decay recipe.

`lr_multiplier(step: int, total_steps: int, warmup_steps: int, power: float = 1.0) -> float`

Returns a pure LR multiplier (not an absolute rate): linear warmup from just above 0 to 1.0 over the first `warmup_steps`, then polynomial decay (`power=1.0` is linear, matching this project's use) from 1.0 back to 0 over the remaining steps.

`apply_lr(optimizer, peak_lrs: list[float], multiplier: float) -> None`

Sets each optimizer parameter group's current learning rate to `peak_lr * multiplier`, iterating the groups in the same order `build_param_groups` constructed them. Called every training step to advance the warmup/decay schedule.

10 pipeline/trainer.py

The shared PyTorch training-and-evaluation engine for all SegFormer runs — the B2 teacher, the B0 direct student, and the B0 distilled student. YOLO training is deliberately handled entirely separately by Ultralytics' own trainer (scripts 06/08), since YOLO uses its own optimizer, LR schedule, and data format; this module makes no attempt to unify the two.

`_load_temperature(teacher_dir: Path) -> float`

Reads a calibrated temperature scalar from `teacher_dir/temperature.json` (written by `02_calibrate_teacher.py`). If the file is missing, logs a warning and returns the safe default $T = 1.0$ (no scaling) rather than raising — an uncalibrated teacher is still usable, so a missing calibration file is not treated as fatal.

`load_teacher(teacher_dir: Path, pretrained: str, device: torch.device, amp: bool = True) -> Callable`

Loads the teacher's best checkpoint (raising `FileNotFoundError` with an actionable message if missing) with `weights_only=True` (a security-hardening choice avoiding arbitrary pickle code execution), sets `.eval()`, loads its calibrated temperature, and returns a `teacher_fn(x)` closure rather than the raw model — so the training loop calls the teacher without knowing anything about its architecture, and a different teacher implementation could be swapped in later without touching the training loop. Inside the closure, the forward pass runs under `torch.no_grad()` (the teacher is frozen; the docstring calls this the single largest memory saving in the distillation setup) and matching AMP settings, returning temperature-scaled sigmoid probabilities.

`evaluate(model: nn.Module, loader: DataLoader, device, threshold: float, amp: bool = True) -> tuple[pd.DataFrame, dict]`

Runs a full no-grad pass over a DataLoader, producing per-image rows (via `metrics.compute_image_row`) and a dataset summary (via `metrics.summarize`). Logits are explicitly cast to fp32 before sigmoid to avoid overflow near float16's representable range. Training always evaluates at a fixed 0.50 threshold — sweeping thresholds every epoch would multiply evaluation cost roughly 19-fold and add noise to the tracked training curve; the true best threshold is determined once, after training, in a separate sweep.

`_threshold_sweep(model, loader, device, thresholds: list[float], amp) -> tuple[pd.DataFrame, float]`

Runs `evaluate()` once per candidate threshold, returning the full sorted table and the single best (highest mean Dice) threshold. This sweep must run against the **validation** set, never test — choosing a threshold from test data would leak test information into model selection and inflate the reported score.

`_train_one_epoch(model, loader, optimizer, sup_loss, dist_loss, scaler, accum_steps, device, clip_grad, teacher_fn, total_steps, warmup_steps, peak_lrs, poly_power, global_step, desc) -> tuple[float, int]`

Runs one training epoch with gradient accumulation, mixed precision, and the warmup/decay schedule interleaved. The teacher (if present) is always run *before* the student under its own no-grad context: this ordering, kept deliberately consistent with `find_optimal_micro_batch`'s own probe ordering, means the teacher's activations are freed before the student's backward graph is

built, roughly halving peak VRAM versus running the student first. Loss is divided by `accum_steps` before backpropagation so accumulated gradients sum to the same magnitude a full-size batch would produce; the optimizer step (with gradient clipping) only fires on accumulation-boundary steps or the epoch's final batch, so a partial trailing window still gets applied rather than silently dropped.

`_build_optimizer_and_schedule(model, cfg: dict) -> tuple[torch.optim.Optimizer, list[float]]`

Builds the AdamW optimizer from `scheduler.build_param_groups` and extracts each group's peak learning rate for the warmup/decay schedule.

`_compute_schedule_params(n_batches: int, accum_steps: int, cfg: dict) -> tuple[int, int, float]`

Computes total optimizer-update steps and warmup steps in terms of actual gradient-update steps (not raw micro-batches or epochs), since the SegFormer paper's schedule is inherently defined against optimizer steps and an epoch-granularity approximation would be imprecise whenever accumulation does not evenly divide the epoch.

`_save_run_config(...) -> None`

Serializes a comprehensive dictionary of hyperparameters and derived quantities to `run_config.json`, so downstream evaluation scripts know exactly how a given model was trained without re-parsing the original CLI invocation.

`_load_resume_state(...) -> tuple`

/

`_save_resume_state(...) -> None`

Save/restore a full training-resume checkpoint (model, optimizer, and AMP scaler state, plus epoch, best score, patience counter, history, and batch-size probe results) every epoch, unconditionally — because GPU sessions can disconnect mid-training, and writing every epoch bounds lost work to at most one epoch. This is deliberately decoupled from `best_model.pt`: the resume state tracks "where training left off," while the best-model file tracks "the best weights found so far," since the most recent epoch may not be the best one.

`train_pytorch(model, model_name, run_dir, train_df, val_df, cfg, device, teacher_fn=None, alpha=0.75) -> dict`

The top-level orchestrator for one full SegFormer run (teacher, direct, or distilled — distinguished purely by whether `teacher_fn` is `None`). Probes or resumes batch sizing, builds DataLoaders and the optimizer/schedule, writes `run_config.json`, then loops epochs calling `_train_one_epoch` and `evaluate`, tracking the best validation Dice, saving `best_model.pt` whenever it improves, writing a resume checkpoint every epoch regardless, and stopping early once the patience budget is exhausted. After training, reloads the *best* (not last-epoch) weights, runs the post-training threshold sweep, and writes `threshold_search.csv` and `val_summary.csv`. Returns the full run summary dictionary.

11 pipeline/yolo_threshold_temp.py

Temperature scaling and threshold sweeping for YOLO26n-sem. Ultralytics' own `.predict()` *converts raw logits straight to a hard class map via argmax internally, exposing no probability map or threshold control — this module bypasses that entirely by calling the underlying* `nn.Module` *directly. For a 2-class semantic head, BCE training pushes logits toward* $\pm\infty$*, producing a near-binary probability histogram with almost nothing near 0.5; temperature scaling (dividing logits by* $T > 1$ *before sigmoid) spreads that histogram back out so a threshold sweep has something meaningful to differentiate.*

```
yolo_raw_class_logits(yolo_model, x: torch.Tensor, out_hw: tuple[int, int]) -> torch.Tensor
```

Runs YOLO's raw `nn.Module` directly (bypassing `.predict()` and its internal argmax), and resizes the raw `[B, nc, h, w]` output to `out_hw` if it does not already match — this is the exact call site where the 640x640 resize happens inside `benchmark_640.py`'s timed section.

```
bruise_prob_from_logits(class_logits: torch.Tensor, temperature: float = 1.0) -> torch.Tensor
```

Converts `[B, nc, H, W]` class logits into a single `[B, H, W]` bruise probability map: for a 2-class head, takes the class-logit difference (bruise minus background) and applies sigmoid — mathematically identical to 2-class softmax, and directly analogous to SegFormer's single-channel sigmoid. Dividing by `temperature` before sigmoid is the de-saturation step; $T = 1.0$ is the model's native (near-binary) output, $T > 1.0$ spreads the histogram back toward 0.5.

```
evaluate_yolo_raw(yolo_model, loader, device, threshold: float, temperature: float) ->
tuple[pd.DataFrame, dict]
```

Same shape/contract as `trainer.evaluate()`, but for YOLO's raw `nn.Module` plus this module's own sigmoid/threshold postprocessing instead of Ultralytics' `.predict()` plus argmax.

```
threshold_temperature_sweep(yolo_model, loader, device, thresholds: list[float], temperatures:
list[float]) -> pd.DataFrame
```

A joint grid sweep over threshold x temperature on validation, scored by mean Dice — the same simple grid-sweep method used for SegFormer, just crossed with an extra temperature axis. Returns the full grid (best-first) so the empty-gap/de-saturation pattern is visible in the saved CSV, not only the single winning row.

```
run_threshold_search(weights_path: str, val_df, cfg: dict, device, run_dir: Path) ->
tuple[pd.DataFrame, float, float]
```

The full validation-side pipeline for one YOLO run: loads `best.pt`, extracts the raw `nn.Module`, sweeps temperature x threshold on validation, saves `threshold_search.csv` (same filename convention as SegFormer, so downstream scripts read both uniformly), and returns the full grid plus the best threshold and temperature.

```
load_yolo_threshold_temp(run_dir: Path) -> tuple[float, float]
```

Reads the best validation `(threshold, temperature)` pair from `threshold_search.csv`, defaulting temperature to 1.0 for any older CSV written before temperature scaling existed.

`load_yolo_model(model_name: str, paths: dict, device: torch.device) -> tuple[nn.Module, float, float, Path]`

Loads a trained YOLO run's raw `nn.Module` (deep-copied out of the Ultralytics wrapper) plus its calibrated `(threshold, temperature)` pair. Returning the raw module (not the wrapper) means a caller can never accidentally fall back to Ultralytics' own argmax postprocessing.

12 pipeline/benchmark_640.py

A device-agnostic (CPU or CUDA) controlled 640x640 deployment-latency benchmark. This exists because `scripts/29_benchmark_inference_all_models.py` raises immediately without CUDA — reasonable for final GPU numbers, but it means the benchmark cannot be exercised at all while the cluster is down. Every timing helper here is device-agnostic: synchronization is a no-op off CUDA, peak memory is reported as `NaN` off CUDA, and fp16 is skipped (not attempted) off CUDA. It also fixes a real inconsistency in script 29: YOLO is preprocessed with the exact same ImageNet-normalized transform (`pipeline.data.get_augmentation`) as SegFormer, matching how YOLO's threshold and temperature were actually calibrated (`yolo_threshold_temp.run_threshold_search` and the script that produced the project's reported test-set Dice both go through `BruiseDataset`, which normalizes with ImageNet mean/std) — script 29's own YOLO preprocessing instead used a plain 0–1 pixel scale, a different input distribution than the one its own loaded threshold was calibrated against.

`class SourceImage (frozen dataclass): stem: str, path: str`

A benchmark image *reference* only — no decoded pixels. Deliberately not preloaded: each fixed-test image is roughly 4022x6024x3 pixels (≈72MB decoded), and holding all 185 of them in memory at once needs about 13GB, which actually crashed with an out-of-memory error on this laptop mid-preload.

`read_source_image(source: SourceImage) -> np.ndarray`

Decodes one image (OpenCV read, BGR to RGB) on demand, called fresh immediately before each timed inference and never cached — keeping peak memory to roughly one image at a time while keeping disk decode excluded from the timed measurement, exactly as before the memory fix.

`class BenchmarkSummary (dataclass)`

The schema of one row of the output summary CSV: model identity, checkpoint, device, precision, threshold, temperature, parameter counts, image/repeat counts, mean/median/p90/p95/std latency, FPS from mean and from median, peak GPU memory, and a fixed description of exactly what was timed.

`class Fixed640Adapter`

The common base class for the controlled benchmark. `__init__(...)` stores the model, threshold, temperature, device, and precision, and builds `self.transform` via `get_augmentation(training=False, ...)` — the identical transform `BruiseDataset` uses at evaluation time. The `autocast_enabled` property is `True` only for fp16 on a CUDA device (there is no fp16 speed benefit to measure on CPU in this codebase). `parameter_count` returns `count_params(self.model)[0]`. `_preprocess(image_rgb)` applies the transform, adds a batch dimension, and moves the tensor to the device. `infer_full(image_rgb)` is abstract: it must take an RGB image already in RAM and return a CPU `uint8` binary mask of shape 640×640; subclasses implement only the one line that differs between architectures.

`class SegFormer640Adapter(Fixed640Adapter)`

`infer_full` preprocesses, runs the model under autocast (no extra resize needed — `SegformerWrapper` already upsamples logits to input resolution internally), divides by temperature (always 1.0 for SegFormer in this project) before sigmoid, thresholds, and returns the CPU `uint8` mask.

`class YOLOSemantic640Adapter(Fixed640Adapter)`

`infer_full` preprocesses through the *same* ImageNet-normalized transform as SegFormer, calls `yolo_raw_class_logits` (never Ultralytics' `.predict()`), applies `bruise_prob_from_logits` with the calibrated temperature, thresholds, and returns the mask.

`build_adapter(spec: dict, paths: dict, device, precision: str) -> Fixed640Adapter`

Builds one adapter from a `configs/benchmark_640_models.yaml` entry, dispatching on family and calling `load_segformer_model` or `load_yolo_model` — deliberately reusing the exact same checkpoint/threshold resolution logic script 29 uses, so "which checkpoint" and "which threshold" can never drift between the two benchmark scripts. Raises `ValueError` for an unrecognized family.

`load_fixed_test_images(paths: dict, max_images: int | None) -> list[SourceImage]`

Lists the fixed test-set images to benchmark (stems and paths only, per `SourceImage`'s memory rationale above). Raises `RuntimeError` if the manifest yields zero images.

`_synchronize(device: torch.device) -> None`

A no-op unless the device is CUDA, in which case it calls `torch.cuda.synchronize`. GPU work is dispatched asynchronously, so without this, a timer would measure kernel-launch latency rather than actual compute time; CPU execution is already synchronous, so nothing is needed there.

`_summarize(values_ms) -> dict[str, float]`

Computes mean, median, p90, p95, standard deviation, and FPS-from-mean/ FPS-from-median from a list of per-call latencies. Raises `ValueError` on an empty input.

`benchmark_adapter(adapter, images, *, warmup: int, repeats: int) -> tuple[BenchmarkSummary, pd.DataFrame]`

The core timing loop. Refuses outright (raises `ValueError`) if fp16 is requested off a CUDA device, rather than silently timing fp32 under an "fp16" label. Runs `warmup` untimed iterations first (absorbing first-call costs like lazy kernel compilation), asserting every returned mask is exactly 640×640 `uint8` or raising immediately. Then, for `repeats` passes over every image: decodes the image fresh (before the timer starts), synchronizes, starts the timer, calls `adapter.infer_full`, synchronizes again, and records the elapsed milliseconds, re-checking shape/dtype every single iteration. Finally computes `_summarize` and peak GPU memory (`NaN` off CUDA), and assembles the `BenchmarkSummary`.

`run_benchmark_640(*, paths_config, common_config, models_config, output_dir, device_name, precisions, max_images, warmup, repeats) -> pd.DataFrame`

The top-level orchestrator called by `scripts/31_benchmark_640_local.py`. Loads and validates configs; refuses to run if `common_train.yaml`'s image size is not 640×640 (this module is hardcoded

to the 640 protocol, and would rather fail loudly than silently benchmark at the wrong resolution); resolves the device (raising if CUDA was requested but unavailable); loads the benchmark images once. For every requested precision and every enabled model in the registry, it builds an adapter and benchmarks it, catching any exception broadly (not just a missing file) so one model failing to load — a missing run folder, or an incompatible checkpoint, per `_pick_matching_checkpoint` above — prints a `SKIP` message and moves on instead of aborting the entire run, matching script 29's own SKIP behavior. After every model, memory is explicitly freed before the next one loads. Finally, writes the summary CSV/JSON, per-image latency CSV, and a metadata JSON recording every config path and setting used for that run.

Module constant: `IMAGE_SIZE = 640`.

Part II

Pipeline Scripts (`scripts/`)

13 scripts/00_build_split.py

The pipeline's foundational stage: builds a fresh, leakage-checked subject-level train/val split, excluding any subject that appears in the fixed test manifest. Splitting at the subject *level (not image level) matters because one forensic subject can contribute multiple images (different lighting/angle) — an image-level split could put the same person's images in both train and val, leaking skin-tone and bruise-appearance information and inflating reported Dice. Building the split fresh here (rather than reusing one from another experiment) keeps every pipeline run self-contained and avoids silently importing another experiment's seed or contamination.*

`main()`

Reads and normalizes both the train and fixed-test manifests, computes the candidate subject pool (train subjects minus test subjects), shuffles with a seeded RNG, splits off `val-fraction` of subjects into validation (guaranteeing at least one), and performs an explicit three-way leakage check (train/val, train/test, val/test), raising `RuntimeError` immediately if any overlap is found — treated as fatal since silent leakage would invalidate every downstream Dice number. Writes `train_val_split.csv` and `split_summary.csv`.

- `--paths` — default `configs/paths.yaml`
- `--val-fraction` — float, default 0.18
- `--seed` — int, default 42

14 scripts/01_train_segformer_b2_teacher.py

Trains the SegFormer-B2 teacher (27.3M parameters) from an ImageNet-pretrained backbone with a randomly initialized binary decode head, using the paper's layer-wise LR recipe. Must run before any distillation step, since those steps consume this teacher's soft labels — a weaker teacher caps the ceiling of every student.

`main()`

Skips training if `best_model.pt` already exists (unless `--force-retrain`). Loads the train/val split, builds the model, and delegates the entire training loop to `pipeline.trainer.train_pytorch` with `teacher_fn=None` (this model has no teacher of its own).

- `--paths, --common` — config paths
- `--force-retrain` — flag; re-train even if a checkpoint exists

15 scripts/02_calibrate_teacher.py

Applies temperature scaling (Guo et al. 2017) to the trained teacher's logits, fitting a single scalar temperature via L-BFGS on validation-set logits. Necessary because BCE-trained logits saturate

toward $\pm\infty$, making soft labels nearly indistinguishable from hard ground truth and defeating the purpose of distillation; dividing by $T > 1$ spreads the distribution so students can see the teacher's real uncertainty near decision boundaries.

`_collect_val_logits(model, loader, device) -> tuple[torch.Tensor, torch.Tensor]`

Runs one no-grad pass over the full validation loader, concatenating all logits and targets. Collected all at once (not streamed per L-BFGS iteration) because L-BFGS needs a stable full-dataset loss/gradient at every step, and the validation set is small enough that this is cheap and done only once.

`_calibrate(logits, targets) -> float`

Optimizes a scalar `log_t` via `torch.optim.LBFGS`, minimizing BCE-NLL of `logits/T` against targets; optimizing in log-space constrains $T > 0$ automatically. Returns $T = \exp(\log\_t)$.

`main()`

Skips if `temperature.json` already exists; raises if the teacher checkpoint is missing. Collects validation logits, computes NLL before and after calibration, writes both to `temperature.json`, and logs a warning if the fitted $T < 1.0$ (unusual — would imply an under-confident model, atypical for BCE training).

- `--paths, --common` — config paths

16 scripts/03_train_segformer_b0_direct.py

Trains the small SegFormer-B0 student (3.7M parameters) directly on ground truth only — no teacher, no distillation — using identical hyperparameters to the teacher. This is the baseline Track A's apples-to-apples comparison measures distillation (script 05) against: since both use identical recipes, any Dice improvement can be attributed specifically to the teacher signal, not a confounded hyperparameter difference.

`main()`

Structurally identical to script 01's `main()`: builds the B0 model and calls `train_pytorch(..., teacher_fn=None)`.

- `--paths, --common` — config paths
- `--force-retrain` — flag

17 scripts/04_optuna_alpha_segformer_b0.py

Runs a TPE (Tree-structured Parzen Estimator) Bayesian search via Optuna to find the optimal alpha (the ground-truth-versus-teacher weighting) for SegFormer-B0 distillation. TPE is used instead of a grid search because it concentrates sampling in the high-Dice region and typically converges faster. Each trial trains only a short number of epochs, since the goal is relative ranking *of alpha values, and rankings stabilize well before full convergence.*

`_run_trial(trial, train_df, val_df, search_cfg, paths, device, teacher_fn, out_dir) -> float`

Suggests an alpha from the search space, trains a **fresh** model for every trial (avoiding contamination from a previous alpha's partially-adapted weights), and returns the resulting mean Dice. Returns 0.0 on any exception (logged) so one failed trial does not abort the whole study; frees GPU memory after every trial.

`main()`

Overrides epochs/patience to a short search-specific value (with patience equal to epochs, so early stopping cannot bias short trials), creates an Optuna study with a seeded TPE sampler and SQLite storage (resumable if interrupted), runs the configured number of trials, and writes the best alpha (falling back to a configured default only if Optuna's result is somehow missing the key) plus the full trial history.

- `--paths, --common` — config paths (search bounds/trial count come from `common_train.yaml`)

18 scripts/05_train_segformer_b0_distilled.py

Trains the final, full-length distilled SegFormer-B0 checkpoint using knowledge distillation from the calibrated teacher at the Optuna-found alpha. Feeds directly into Track A/B evaluation.

`_load_optuna_alpha(project_root, cfg) -> float`

Reads the Optuna best-alpha CSV, raising `RuntimeError` (never silently defaulting) if it is missing — a silent default would misrepresent results as using an Optuna-optimized alpha when they did not.

`main()`

Resolves alpha from a CLI override or the Optuna result, loads the calibrated teacher, builds a fresh B0 model, and calls `train_pytorch(..., teacher_fn=teacher_fn, alpha=alpha)`.

- `--paths, --common` — config paths
- `--alpha` — float, optional override of the Optuna-found value
- `--force-retrain` — flag

19 scripts/06_train_yolo_sem_direct.py

Trains YOLO26n-sem directly (no distillation) using Ultralytics' own training pipeline rather than the custom SegFormer trainer — a deliberate choice since Ultralytics tightly couples its backbone/neck/head training with its own AMP, EMA, and augmentation machinery, and extracting the raw `nn.Module` for a custom loop would risk losing that stability.

`build_split(df, out_dir, split) -> None`

Builds the Ultralytics-format dataset for one split: symlinks each source image (avoiding full copies of large forensic images) and converts each binary ground-truth mask into a class-index PNG, since Ultralytics' semantic task expects class-index masks, not confidence floats.

`_write_data_yaml(run_dir, data_dir, train_n, val_n) -> Path`

Writes the Ultralytics `data.yaml` descriptor Ultralytics' training API requires, which also serves as a permanent record of exactly what data trained the model.

`main()`

Builds the dataset (skipping rebuild if already complete), then trains (or resumes, or skips if already trained) via Ultralytics' own `.train()` call with `task="semantic"`, 640 image size, cosine LR, and mosaic augmentation disabled for the final epochs (so the model fine-tunes on single-image crops matching test-time input). Afterward evaluates on validation using Ultralytics' native `.predict()` (argmax, no threshold/temperature tuning) and writes the summary CSVs.

- `--paths, --common` — config paths
- `--force-rebuild-dataset, --force-retrain` — flags

20 scripts/07_optuna_alpha_yolo_sem.py

The analogous Optuna alpha search for YOLO, but using offline *knowledge distillation: Ultralytics' training loop has no hook for injecting custom per-batch soft targets, so fused pseudo-masks (ground truth blended with the temperature-scaled teacher probability, weighted by alpha and thresholded) must be precomputed to disk before each trial trains.*

`build_split(df, out_dir, split, alpha, teacher, temperature, cfg, device) -> None`

For the training split with a teacher present: computes the teacher's probability map per image, fuses it with ground truth by alpha, and thresholds the fused mask. For the validation split, always uses pure (unfused) ground truth, since validation masks are the scoring reference and must not be softened.

`run_trial(alpha, trial_number, weights_path, out_dir, train_df, val_df, cfg, teacher, temperature, device) -> float`

Builds a fresh pseudo-mask dataset per trial, trains a short YOLO run (early stopping disabled, warmup capped, logging suppressed), evaluates on validation via Ultralytics' native prediction, and returns mean Dice. Deletes the trial's dataset afterward (retaining only the Ultralytics run logs) to avoid exhausting disk space across many trials.

`main()`

Loads the raw teacher `nn.Module` directly (not via the `teacher_fn` wrapper, since the pseudo-mask builder needs to run its own preprocessing on raw images), creates a seeded TPE Optuna study, runs the configured trial count, and writes the best-alpha CSV and trial history.

- `--paths, --common` — config paths

21 scripts/07b_threshold_yolo_direct.py

Performs the joint (temperature, threshold) grid sweep on validation for the direct *YOLO model, bypassing Ultralytics' native prediction path entirely. All sweep logic lives in the shared `pipeline/yolo_threshold_temp.py` helper, not duplicated here.*

`main()`

Raises if the direct YOLO weights are missing; skips if `threshold_search.csv` already exists (unless `--force-rerun`); calls `run_threshold_search` and logs the best pair plus the top rows of the grid.

- `--paths, --common` — config paths
- `--force-rerun` — flag

22 scripts/07c_threshold_yolo_distilled.py

Identical logic to 07b, applied to the distilled *YOLO checkpoint. Kept as a separate script (rather than reusing 07b's result) because distillation changes the model's learned decision boundary and logit scale, so the jointly optimal (temperature, threshold) pair typically differs from the direct model's even with an identical architecture.*

`main()`

Structurally identical to 07b's, pointed at the distilled run directory.

- `--paths, --common` — config paths
- `--force-rerun` — flag

23 scripts/08_train_yolo_sem_distilled.py

Trains the final, full-length distilled YOLO checkpoint using the Optuna-found alpha from script 07, following the same offline pseudo-mask distillation approach.

`_load_optuna_alpha(project_root) -> float`

Reads the YOLO Optuna best-alpha CSV, raising if missing (mirroring script 05's rationale — no silent default).

`build_split(df, out_dir, split, alpha, teacher, temperature, cfg, device) -> None`

Same fusion logic as script 07's version, wrapped in a progress bar since building pseudo-masks requires one GPU forward pass per training image.

`main()`

Resolves alpha, builds/rebuilds the pseudo-mask dataset (freeing the teacher from GPU memory before YOLO training starts), trains via Ultralytics with the same recipe as script 06, evaluates on validation with native prediction, and records alpha in `run_config.json`.

- `--paths, --common` — config paths
- `--alpha` — float, optional override
- `--force-rebuild-dataset, --force-retrain` — flags

24 scripts/09_evaluate_test.py

A fast "sanity check" evaluation of all 5 models on the fixed held-out test set — basic Dice/IoU/Precision/Recall and simple FPS timing, no bootstrapping or statistical tests, distinct from the rigorous Track A/B scripts. YOLO here uses Ultralytics' native argmax prediction (no threshold/temperature), contrasted against script 09b's temperature-scaled version.

`_load_best_threshold(run_dir) -> float`

Reads a SegFormer run's validation-selected threshold; raises rather than defaulting to 0.5, so results stay comparable to the Track A/B scripts, which always use the validation-selected threshold.

`eval_pytorch(run_name, pretrained, run_dir, test_df, cfg, device, out_dir) -> dict`

Loads the model and threshold, evaluates the fixed test set at batch size 1 (chosen so per-image timing is accurate rather than amortized across a batch), timing each forward pass with GPU synchronization, and aggregates per-image metrics.

`eval_yolo(run_name, run_dir, test_df, cfg, out_dir) -> dict`

Evaluates YOLO via Ultralytics' native `.predict()` (argmax, no threshold), timing with wall-clock `time.perf_counter()` since `.predict()` is already a blocking call that includes any GPU wait.

`main()`

Iterates the 3 SegFormer runs (skipping any without a checkpoint) calling `eval_pytorch`, then the 2 YOLO runs calling `eval_yolo`, and writes a combined, Dice-sorted comparison CSV.

- `--paths, --common` — config paths
- `--force` — flag; re-evaluate even if outputs exist

25 scripts/09b_evaluate_yolo_temp_scaled.py

Re-evaluates the two YOLO models on the fixed test set using the validation-selected (temperature, threshold) pairs, since Ultralytics' `.predict()` pipeline hard-codes an internal argmax step that cannot be intercepted to inject temperature scaling. Outputs are written to a parallel directory so both this script's and script 09's results are preserved for comparison.

`_load_temp_threshold(run_dir) -> tuple[float, float]`

Reads the validation-selected `(threshold, temperature)` pair, raising if the sweep has not been run — falling back to (T=1, threshold=0.5) would be misleading, since YOLO's near-binary logits specifically need $T \gg 1$ for a threshold to be meaningful.

`eval_yolo_with_temp_threshold(run_name, run_dir, test_df, cfg, device, out_dir) -> dict`

Deep-copies the raw `nn.Module` out of the Ultralytics wrapper (to avoid mutating the wrapper's internal state), evaluates using `yolo_raw_class_logits` and temperature-scaled `bruise_prob_from_logits` directly, bypassing Ultralytics' postprocessing.

`main()`

For each YOLO run, skips (with a warning, since this script is supplementary) if the threshold sweep has not been run; otherwise evaluates and writes a combined comparison CSV.

- `--paths, --common` — config paths
- `--force` — flag

26 scripts/10_consolidate_results.py

A lightweight, no-GPU-work aggregation script merging each model's validation and (if available) test summaries into one wide CSV alongside training-recipe metadata — purely for reporting convenience, distinct from the deeper statistical Track A/B outputs.

`main()`

For each of the 5 fixed run names, reads `val_summary.csv` (skipping with a message if absent), splits columns into identity/recipe fields versus metric fields, optionally merges in `test_summary.csv`, and writes the combined table.

- `--paths` — config path

27 scripts/10_track_a_evaluate.py

"Track A": a strict, apples-to-apples benchmark of only the three SegFormer variants trained under identical conditions, styled after Gut et al. 2022. YOLO is deliberately excluded here because it uses a fundamentally different optimizer, schedule, loss, and data format — including it would violate the "identical conditions" premise (that comparison is Track B's job). Reports Dice/IoU/Precision/Recall, boundary metrics, fairness breakdowns, three categories of inference speed, and statistical rigor via bootstrap confidence intervals and Wilcoxon tests against the B0-direct baseline.

`load_best_threshold(run_dir) -> float`

Same pattern as other scripts — reads the validation-selected threshold, raising rather than silently defaulting.

`_gpu_timed(fn, n_warmup, n_iters) -> tuple[float, float, float]`

A generic GPU timing helper: warms up untimed, then times `n_iters` calls with `cuda.synchronize()` bracketing each, since GPU work is asynchronous and an unsynchronized timer would measure kernel-launch latency rather than actual compute time.

`_build_single_image_tensor(image_path, cfg, device) -> torch.Tensor`

Loads one image through `BruiseDataset` and returns a GPU-ready tensor, built outside the timed function since the controlled benchmark measures only the forward pass and postprocessing, deliberately excluding disk I/O (which is not reproducible due to OS disk-cache effects).

`benchmark_speed(model, best_thr, benchmark_img, test_paths, cfg, device, n_warmup, n_iters) -> dict`

Measures three timing categories: `raw_forward` (bare model call, isolating architecture speed), `mask_output` (forward plus sigmoid plus threshold), and `end_to_end` (full realistic pipeline including disk read, preprocessing, GPU transfer, and threshold), the last measured once per real test image to average out disk I/O variance rather than repeating a single cached image.

`_run_inference_loop(model, loader, best_thr, device, amp, surf_dice_delta) -> list[dict]`

One pass over the test loader computing extended per-image metrics (including surface Dice) for every image, keeping per-image rows (not just an aggregate) so later analyses — per-skin-tone breakdowns, Wilcoxon tests, outlier inspection — do not require re-running inference.

`eval_track_a(run_name, pretrained, run_dir, test_df, cfg, device, out_dir, benchmark_img, test_paths, surf_dice_delta=2.0, n_bootstrap=2000, n_warmup=15, n_iters=100) -> dict`

The full per-model pipeline: loads the validation threshold, loads `best_model.pt` (never last-epoch weights, avoiding an overfit checkpoint), runs inference, saves per-image results *before* aggregation (so a crash during aggregation cannot lose raw predictions), computes the extended summary, runs the speed benchmark separately, and returns everything plus the per-image Dice array (used later for Wilcoxon tests, not persisted to CSV).

`_run_wilcoxon_tests(all_results, baseline_name, out_dir) -> list[dict]`

Runs a Wilcoxon test between each non-baseline model's per-image Dice and the baseline's (`segformer_b0_direct`, chosen as the simplest student with no teacher — directly answering "does distillation help?").

`main()`

Iterates the 3 SegFormer runs (reusing cached results if already evaluated unless `--force`), runs the Wilcoxon tests against the baseline, and writes a Dice-sorted comparison CSV plus the Wilcoxon table.

- `--paths, --common` — config paths
- `--n-warmup, --n-iters` — GPU timing warmup/timed iteration counts (default 15/100)
- `--n-bootstrap` — bootstrap resamples for CI (default 2000)
- `--surf-delta` — Surface Dice tolerance in pixels (default 2.0)
- `--force` — flag

28 scripts/11_track_b_evaluate.py

"Track B": a deployment-realistic comparison of all 5 models, each using its own best/realistic recipe rather than a forced-identical protocol. The docstring is explicit that Track B can only support "better in our realistic setup," not "architecturally superior" — that stronger claim requires Track A's controlled comparison, since SegFormer and YOLO use fundamentally different optimizers/losses/preprocessing here.

`load_segformer_threshold(run_dir) -> float`

/

`load_yolo_threshold_temp(run_dir) -> tuple[float, float]`

Same validation-selected threshold (and, for YOLO, temperature) loading as Track A, extended to cover both model families.

`_gpu_timed(fn, n_warmup, n_iters) -> tuple[float, float, float]`

Identical helper to Track A's, for the identical reason (avoiding measurement of async kernel-launch latency instead of real compute time).

`_e2e_times_segformer(model, best_thr, test_paths, cfg, device) -> np.ndarray`

Measures true end-to-end wall-clock latency for SegFormer across every test image: disk read, color conversion, preprocessing, GPU transfer, forward, sigmoid, threshold, and device-to-host transfer, each step timed together.

`_e2e_times_yolo(yolo_nn, best_thr, best_temp, test_paths, cfg, device) -> np.ndarray`

The YOLO analogue, using `yolo_raw_class_logits` and temperature-scaled `bruise_prob_from_logits` in place of a plain sigmoid.

`eval_segformer_track_b(...) -> dict`

Nearly identical to Track A's per-model evaluation, but written under `track_b_evaluation/` and additionally recording parameter count and `temperature_used=1.0`, so the final comparison table can show parameter counts side by side across SegFormer and YOLO.

`eval_yolo_track_b(...) -> dict`

The YOLO analogue: deep-copies the raw `nn.Module`, evaluates with temperature-scaled logits, and runs the same three-category speed benchmark using YOLO-specific logit extraction in place of a plain forward call.

`_print_result(run_name, res) -> None`

A logging helper printing one formatted summary line per model: median Dice with its 95% CI, HD95, complete-miss rate, and both raw-forward and end-to-end FPS.

`main()`

Iterates the 3 SegFormer runs then the 2 YOLO runs (skipping any missing weights or threshold sweep), runs Wilcoxon tests against the B0-direct baseline for every other model, and writes the combined comparison and Wilcoxon tables.

- `--paths, --common` — config paths
- `--n-warmup, --n-iters, --n-bootstrap, --surf-delta, --force` — same meaning as Track A

29 scripts/24_consolidate_all_results.py

A final "gather everything" step copying every human-readable results artifact from scripts 00–23 (plus ad-hoc benchmark/audit scripts) into one self-contained mirrored folder, so results can be

reviewed or zipped without hunting across many scattered directories. Explicitly distinguished from ***10_consolidate_results.py****, which merges/transforms numbers into one CSV; this script only copies files, never transforms anything. Model checkpoint weights are never copied (they would make the folder multi-GB, defeating the point), and a missing source directory is logged as a warning in the output manifest rather than raised as an error, since the script must be runnable at any point in the pipeline's lifetime.*

`_copy_filtered_tree(src_dir, dst_dir, manifest: list[str]) -> None`

Recursively copies only files with an allowed extension (CSV, JSON, TXT, XLSX, LOG) from `src_dir` into `dst_dir`, preserving relative structure; anything else (weights, images, caches) is simply skipped by the extension filter with no special-casing needed.

`_copy_file(src, dst, manifest) -> None`

Copies a single named file, recording a MISSING line in the manifest instead of raising if the source does not exist.

`main()`

Copies the split directory, every per-model run directory's result files, the Optuna search results, every evaluation folder, and the ITA labels directory (if configured) into the output tree, then writes the accumulated `MANIFEST.txt`.

- `--paths` — config path
- `--out-name` — default `CONSOLIDATED_RESULTS`

30 scripts/26_migrate_legacy_segformer_checkpoints.py

A one-time migration utility rewriting SegFormer checkpoint parameter names between two different internal layouts HuggingFace ***transformers*** *has used across versions — discovered 2026-07-07 while running scripts 25 and 12, and confirmed identical across two independently trained checkpoints. Rewriting the checkpoint file's own keys (rather than patching every loading call site in* ***pipeline/trainer.py*** *and every evaluation script) fixes every caller at once without touching a single* ***.py*** *file — and this project's own ground rule forbids modifying* ***pipeline/*** *or scripts 00–12. Defaults to a dry run that verifies the remapped state dict loads cleanly (strict, zero missing/unexpected keys) before ever writing anything; the original file is always backed up first.*

`_remap_key(key: str) -> str`

Applies each regex rule from the module's rename table in order, returning the first match's replacement, or the key unchanged if it is already new-style (the rules are mutually exclusive, so order does not affect correctness).

`_is_legacy_checkpoint(state_dict: dict) -> bool`

Returns `True` if any key still contains an old-style substring exclusive to the legacy layout.

`_migrate_one(model_name, run_dir, pretrained, apply: bool) -> None`

Loads the checkpoint, skips (with a log message) if it is missing or already new-style, otherwise remaps every key, then **verifies** the remap by building a fresh model and diffing key sets — raising rather than trusting an unverified remap — before ever writing. Only writes (backing up the original first, and only if no backup already exists) when `apply=True`.

`main()`

Resolves which model names to migrate (default: the three known-affected checkpoints), picks the correct pretrained-weights key per model, and calls `_migrate_one` for each.

- `--paths` — config path
- `--model-name` — repeatable; defaults to the 3 known-affected models
- `--apply` — flag; without it, dry-run only

31 scripts/27_evaluate_test_640_models_01_to_11.py

Computes test-set Dice (and extended metrics) for the five original models always at a fixed 640x640 resolution — ground truth is resized down by the data loader, and predictions are never *upscaled back to native camera resolution for comparison. This avoids the internal upscale step that Ultralytics' own prediction path performs, giving a leaner "what is the Dice at 640, no upscaling" number than the fuller Track A/B scripts.*

`_load_threshold(run_dir) -> float`

/

`_load_threshold_temp(run_dir) -> tuple[float, float]`

The same validation-selected threshold (and, for YOLO, temperature) loading pattern used throughout the project, raising rather than defaulting silently.

`_score_rows(rows, out_dir, run_name, best_thr) -> None`

Writes the per-image CSV, computes the extended summary (2000 bootstrap resamples), attaches run identity and an explicit note documenting the no-upscale evaluation protocol, and writes the summary CSV.

`_evaluate_segformer(model_name, pretrained_key, paths, cfg, device, test_df, out_root) -> None`

Evaluates one SegFormer model over the test set at batch size 4 under autocast, applying sigmoid and the validation threshold, and scoring with extended per-image metrics.

`_evaluate_yolo(model_name, paths, cfg, device, test_df, out_root) -> None`

Evaluates one YOLO model using `yolo_raw_class_logits` resized to match the batch's own 640 tensor size (never native resolution) plus temperature-scaled probabilities — bypassing Ultralytics' `.predict()` entirely.

`main()`

Loops `_evaluate_segformer` over the 3 SegFormer models and `_evaluate_yolo` over the 2 YOLO models.

- `--paths, --common` — config paths

Module constants: `SURF_DICE_DELTA = 2.0`, `N_BOOTSTRAP = 2000`, `EVAL_BATCH = 4`.

32 scripts/29_benchmark_inference_all_models.py

A GPU-only inference speed benchmark across every white-light-deployable model, reusing the deployment-style timing logic later ported (and corrected) into ***pipeline/benchmark_640.py****. Reports* ***raw_forward_fps*** *(bare model call, isolating architecture speed) and* ***full_pipeline_fps*** *(camera image in RAM through to a mask resized back to the* original *camera resolution — the real deployment speed). Excludes disk read, model loading, and warmup from timing. Note: this script's YOLO preprocessing scales input to a plain 0–1 range with no ImageNet normalization, which* ***pipeline/benchmark_640.py****'s docstring identifies as inconsistent with how YOLO's threshold and temperature were actually calibrated.*

`load_image_raw(image_path: str) -> tuple[np.ndarray, tuple[int, int], str]`

Reads and BGR-to-RGB-converts an image, returning it with its native `(height, width)` and filename stem. "Disk read is setup, not part of timed camera pipeline."

`make_segformer_input(img_rgb, size, device) -> torch.Tensor`

Resize plus ImageNet normalize plus to-tensor — "same inference preprocessing structure as BruiseDataset for SegFormer."

`make_yolo_input(img_rgb, size, device) -> torch.Tensor`

Resize plus to-tensor, then manually divided by 255 — *not* ImageNet-normalized, the inconsistency noted above.

`benchmark_gpu_fn(fn, device, warmup, n_iters) -> tuple[float, float, float]`

Generic GPU timing harness: warms up untimed, times `n_iters` calls individually with synchronization bracketing each, returns mean/std/FPS.

`dtype_configs(include_fp32, include_fp16)`

Returns the list of `(name, dtype, use_amp)` tuples to benchmark, depending on which precisions were requested.

`benchmark_segformer_one_dtype(...) -> tuple[dict, list[dict]]`

Times a bare `raw_forward` once on a warm input, then for *every* benchmark image times a `full_pipeline` closure that re-preprocesses, forwards, thresholds, and resizes the binary mask back to that image's original resolution — this per-image resize-back is the defining feature of this script's "full pipeline" measurement (contrast with `pipeline/benchmark_640.py`, which keeps the mask at 640x640 by design).

`benchmark_yolo_one_dtype(...) -> tuple[dict, list[dict]]`

Same structure for YOLO, using `yolo_raw_class_logits` plus temperature-scaled probabilities before thresholding and resizing back to native resolution.

`select_benchmark_images(paths, args) -> list[dict]`

Either benchmarks a single `--image`, or loads the fixed test manifest (optionally shuffled by a fixed seed) and takes the first `--n-images` rows, reading each one fully into memory as a record.

`main()`

Raises `RuntimeError` immediately if no CUDA device is available — "CPU FPS is not meaningful here." Loads/validates configs, selects benchmark images, then loops the SegFormer and YOLO model lists (each optionally filtered by `--models`), loading each (skip-with-warning on failure) and running every resolution/dtype combination, writing the summary and per-image CSVs.

- `--paths, --common` — config paths
- `--out, --per-image-out` — output CSV paths
- `--image, --n-images, --sample-seed` — image selection (default 20 images, seed 2026)
- `--resolutions` — default `[640]`
- `--n-iters, --warmup` — default 200/20
- `--fp32, --fp16` — flags; both run if neither given
- `--models` — optional subset filter

33 scripts/31_benchmark_640_local.py

A thin CLI wrapper around `pipeline.benchmark_640.run_benchmark_640()` — the controlled 640x640 benchmark that runs on CPU or CUDA, unlike script 29 which hard-requires CUDA. `--device` defaults to auto-detection (CUDA if available, else CPU) rather than raising, so it actually runs on this laptop. Nearly all logic is deferred to `pipeline/benchmark_640.py`; this file only parses arguments and forwards them.

`default_device() -> str`

Returns `"cuda:0"` if available, else `"cpu"`.

`parse_args() -> argparse.Namespace`

Defines the CLI surface (below).

`main()`

Forwards every parsed argument directly into `run_benchmark_640`.

- `--paths, --common, --models` — config paths (default `configs/paths.yaml`, `configs/common_train.yaml`, `configs/benchmark_640_models.yaml`)
- `--out-dir` — default `results/benchmark_640_local`
- `--device` — default: auto-detected
- `--precision` — choices `fp32`/`fp16`, default `fp32`; fp16 auto-skipped off CUDA
- `--max-images` — default 185 (the full fixed test set)

- `--warmup, --repeats` — default 30/3

34 scripts/fix_local_manifest_paths.py

A one-off utility rewriting hardcoded ORC-cluster Linux absolute paths baked into manifest CSVs' `image_path/mask_path` columns, so they resolve on this laptop. Necessary because `configs/paths.yaml` only controls where the manifest files *are found — it is never consulted again once a manifest is loaded, since `BruiseDataset` reads whatever string sits in those columns and hands it straight to `cv2.imread()`. Only changes where a file is found on disk, never a pixel value or threshold, so it cannot affect any already-recorded evaluation number.*

`remap_one_manifest(csv_path: Path) -> None`

Backs up the untouched original to a sibling `.orc_backup.csv` file (only on first run), then always rebuilds the live CSV fresh from that backup with every known ORC prefix replaced — making the script idempotent and safe to re-run any number of times, since the live file is always a correct rebuild from the one true original, never a second pass over an already-replaced file.

`verify_paths_resolve(csv_path: Path) -> list[str]`

Loads the CSV through `pipeline.data.normalize_manifest` (reusing the real pipeline's own column-normalization logic rather than re-guessing column names) and returns every `image_path/mask_path` value that does not resolve to a real file.

`main()`

For each of the 4 manifests wired into `paths.yaml` (the other manifests in the training/test split and the two ALS manifests, plus the fixed test manifest), skips with a warning if the file does not exist yet, otherwise remaps and verifies it. Raises `RuntimeError` listing every path that still fails to resolve after remapping.

- `--paths` — config path

35 scripts/visualize_segformer_stages.py

Produces a figure visualizing how a trained SegFormer-B2 checkpoint processes one bruise image, stage by stage: input, patch embedding, each of the four encoder stages, the decoded probability heatmap, and the final thresholded mask. The docstring stresses these are the model's actual, real intermediate activations on the given image (captured via a forward hook), not a schematic illustration.

`to_heatmap(feature_map: torch.Tensor, out_h: int, out_w: int) -> np.ndarray`

Averages a feature map across channels, min-max normalizes to $[0, 1]$, and resizes for smooth visualization.

`overlay(rgb_uint8: np.ndarray, heat01: np.ndarray, alpha: float = 0.55) -> np.ndarray`

Converts a normalized heatmap to a colormap image and alpha-blends it over the original photo.

`main()`

Builds the model and loads the checkpoint; registers a forward hook on the first patch-embedding module to capture its raw spatial output; preprocesses the input image by hand (resize, ImageNet normalize); runs one forward pass capturing all four encoder stages' hidden states plus the final upsampled logits; assembles a 7-panel figure (input, patch-embedding overlay, four stage overlays with plain-English captions, the probability heatmap, and the binary mask) and saves it as a PNG.

- `--checkpoint, --pretrained, --image` — required
- `--img-size` — default 640
- `--threshold` — default 0.5
- `--out` — default `stage_visualization.png`

36 scripts/yolo_wl_audit_v1.py

A single-file, deliberately non-modular diagnostic script answering four specific items raised in a stakeholder meeting, for the two white-light YOLO models: (1) FPS at 640 only, with no upsample to native resolution — reversing earlier guidance that required timing the resize-back step; (2) test Dice at 640 with ground truth resized by the data loader, never native resolution; (3) a temperature-scaling sanity check, reporting test Dice at both the swept-best (temperature, threshold) and at temperature 1.0 side by side, so it is visible whether temperature scaling actually helps; and (4) a comparison of the project's own bare-PyTorch-plus-threshold pipeline against Ultralytics' native prediction path on the same test set, at their respective native resolutions (a difference the audit deliberately reports rather than silently normalizing away).

`_make_yolo_input_640(img_rgb, img_h, img_w, device) -> torch.Tensor`

The same structural preprocessing as `BruiseDataset` for YOLO, kept local rather than imported, per this project's convention that standalone utility scripts do not import each other.

`_benchmark_fps_640(yolo_nn, img_rgb, img_h, img_w, best_thr, best_temp, device) -> dict`

Times a raw-forward closure and a full-pipeline closure, both explicitly without any resize back to camera resolution, per item 1 above.

`_evaluate_ultralytics_native(wrapper, test_df, img_h, device_str) -> pd.DataFrame`

Evaluates every test image using Ultralytics' own `.predict()` (its internal argmax, resized to native ground-truth resolution) — deliberately no temperature scaling or threshold control, the "as is" side of item 4.

`_audit_one_model(model_name, paths, cfg, device) -> None`

Runs items 3, 2, 4, and 1 in sequence for one model: searches (temperature, threshold) on validation and isolates the temperature-1.0 baseline; evaluates the test set at 640 under both the swept-best and temperature-1.0 operating points, logging a warning if temperature scaling does not actually help; evaluates Ultralytics' native path at native resolution; benchmarks FPS at 640; and writes one consolidated summary row combining every result.

`main()`

Hard-requires CUDA ("FPS numbers are meaningless on CPU") and loops `_audit_one_model` over the direct and distilled YOLO models.

- `--paths, --common` — config paths

Module constants: `N_WARMUP = 20`, `N_ITERS = 200`.